University San Francisco de Quito

Artificial Intelligence

Name: Raymond Portilla **NRC:** 1651
Banner ID: 00335050 **Date:** 16/09/2026

HOMEWORK #1

Research Question

In this section, I will address my analysis of the discussion surrounding Chess versus Go.

State of the art

Both games have a long story in AI, but they did not move at the same speed. This is the first fact that I find interesting to think about.

For chess, the famous moment was in 1997. IBM Deep Blue beat the world champion Garry Kasparov. Deep Blue was not a learning system like the ones we hear about now. It used a very fast search, alpha-beta pruning, and a big evaluation function written by people (Campbell et al., 2002). After that, chess programs kept this idea for many years. Today the strongest engines, like Stockfish, still search with alpha-beta, but the evaluation is now a neural network. There is also another family, like AlphaZero, that learns from self-play and uses Monte Carlo tree search (Silver et al., 2018).

For Go the story is later. For a long time computers were much weaker than top humans. The board is larger, and it is hard to say who is winning just by counting simple things. The big change was AlphaGo, and then AlphaGo Zero and AlphaZero. They train a deep neural network by playing against themselves, and they use Monte Carlo tree search to choose a move (Silver et al., 2018). Open programs today still follow this same idea.

So, if I have to name the most advanced algorithms, I would say this. In chess, the practical winner is still alpha-beta search plus a learned evaluation. In Go, it is a neural network plus Monte Carlo tree search. AlphaZero is important because the same method can play both games at a very high level, starting from the rules only.

What is similar and what is different?

They are similar in many ways. Both are two-player games with perfect information. There is no hidden card and no luck. Both are zero-sum: if I win, you lose. In both cases the computer builds a game tree and needs some way to evaluate positions. Also, both games are now superhuman. A normal engine on a computer is already stronger than any person.

The differences are more interesting for me. Chess has about 35 legal moves on average. Go has around 250. The full game tree of chess is already huge, about 10^123, but Go is much

larger, about 10^360 (van den Herik et al., 2002). Chess also has a clear material count: a queen is worth more than a pawn, so a handmade evaluation worked early. In Go, the stones all look the same, and a small local change can decide a big area later. This is why alpha-beta plus human knowledge was enough for chess in the 1990s, but it was not enough for Go.

Another difference is training. Old chess programs had a lot of human knowledge inside the evaluation. AlphaZero starts from the rules only and learns by self-play (Silver et al., 2018). That is closer to how modern Go programs work.

Computational complexity

Here I think we have to be careful, because "complexity" can mean two things.

If we talk about the size of the search, chess is already too big to solve by brute force, and Go is even bigger. Van den Herik et al. (2002) give about 10^46 positions for chess and about 10^172 for 19×19 Go. So even storing every position is impossible. This is also why these games are still interesting for AI: we cannot just list all boards.

If we talk about how hard it is to play perfectly, the proofs are for bigger and bigger boards, not only for the normal 8×8 or 19×19 size. Fraenkel and Lichtenstein (1981) show that if we grow the chess board, finding a perfect strategy takes time that grows very fast, in an exponential way with the size of the board. For Go, Lichtenstein and Sipser (1980) show that even deciding who is winning in a given position is already a computationally very hard problem. So both games are hard in theory, not only because the tree is large, but because there is no cheap way to compute a perfect play.

Can we use BFS, DFS, minimax or alpha-beta?

In theory, yes, because a game is a tree of moves. In practice, no, not as the only method.

BFS would look at all positions level by level. It could find a shortest win, but it would use too much memory. DFS would use less memory, but it can go down a very long line and miss a short win. The book already warns about this for search in general (Russell & Norvig, 2021). For chess and Go the tree is so large that plain BFS or DFS cannot finish.

Minimax is the right idea for this type of game. One player wants the maximum score, and the other wants the minimum. But the full minimax tree is the huge game tree I mentioned before, so we cannot build it until the end of the game.

Alpha-beta pruning is a smarter minimax. It cuts branches that will not change the final decision. For chess this worked very well. Deep Blue was basically this idea, only at a huge scale (Campbell et al., 2002), and strong engines today still use it. For Go, alpha-beta alone was not enough. The branching factor is too large, and the evaluation was too weak. That is why modern Go uses Monte Carlo tree search with a neural network: the net says which moves look promising, so the search does not try everything (Silver et al., 2018).

My short answer is this. We can use minimax and alpha-beta as the base, especially in chess, but not plain BFS or DFS. For Go we still search a tree, but the search is Monte Carlo tree search, not classic alpha-beta. The old algorithms did not disappear. They became part of a bigger system.

Practical Questions

1. 8-Tile

I modeled the 8-puzzle like the book does in Section 3.2. A state is a tuple of nine numbers. Zero is the blank. An action slides the blank up, down, left or right. Every slide costs 1, so a shortest path is just the path with the fewest slides.

I used the start board from the book figure of the 8-puzzle:

7	2	4
5		6
8	3	1

The goal is:

1	2	3
4	5	6
7	8	

I also checked inversion parity, because only half of the boards can reach a given goal. My start and goal have the same parity, so a solution exists.

For best-first I used greedy best-first, which means f(n) = h(n). This is not A*, otherwise it would use f(n) = g(n) + h(n). I did it this way because in the book greedy best-first is the simple version that only looks at the heuristic (Russell & Norvig, 2021, Section 3.5).

My free heuristic is misplaced tiles: how many tiles are not in their goal square, not counting the blank. I picked it because Section 3.6 presents it together with Manhattan distance, and it is easy to code and to understand.

I used graph search, not tree search. That means I keep a visited set, so I do not expand the same board twice. The book says this is important, because otherwise the 8-puzzle repeats the same states again and again. I did not draw the full BFS graph, because it has more than one hundred thousand generated nodes, so a picture would not help. Instead I report generated and expanded nodes, which is the size of that graph.

Results

Strategy	Path length	Generated	Expanded
BFS	20	149,026	55,409
DFS	66,056	269,224	99,900
Best-first Euclidean	94	1,132	413
Best-first Manhattan	48	339	123
Best-first misplaced tiles	116	3,069	1,134

BFS is optimal here. All actions have the same cost and, in that case, BFS finds a solution with a minimal number of actions. So 20 is the shortest path my program can claim.

DFS is not optimal, and it is not even short. It went very deep and came back with 66,056 moves. That matches what the book warns: DFS can find a long path even when a short one exists, because it always expands the deepest node first.

The three greedy searches are also not optimal. They do not count g(n), so they can walk away from a short path if the heuristic looks good. Manhattan was the best of the three (48 moves and only 123 expanded nodes). The book says Manhattan dominates misplaced tiles, so this result did not surprise me. Euclidean is a real distance, but tiles cannot move in diagonal, so it is a weaker guide than Manhattan. Misplaced tiles is the weakest: it only says “this tile is wrong”, not how far it is.

I keep BFS as the answer for the shortest path. I keep Manhattan as the heuristic I would use later, because it expanded the fewest nodes among the informed searches.

2. 80-Tile

This is the same idea as the 8-puzzle, but on a 9×9 board. Each tile has a number and a letter for its quadrant (for example 3F). The hole starts in the bottom-right corner, with 9I.

The goal moves the quadrants to:

A B C
H I D
G F E

Each 3×3 is also rotated. After that, the hole sits in the center of the board.

The start is solvable: the permutation parity matches the taxicab parity of the blank. Manhattan distance of the start is 306. That number is already a lower bound. Any real solution has at least 306 moves.

A* with Manhattan is the algorithm that would give the shortest path, because Manhattan never overestimates the remaining slides (Russell & Norvig, 2021, Section 3.6). I ran it with a limit of 250,000 expansions. It did not finish. After that limit the best h was still 287. This is not a bug in the code. A* is slow here because it is optimal: it will not return a path until it can almost prove that no shorter path exists. The true length is much larger than 306, so the search tree is huge. The book also says that larger sliding puzzles get very hard, even with a good heuristic.

So I switched to two methods from the same chapter that give up optimality on purpose, just to actually finish:

Weighted A* with W = 10, so f = g + 10·h. The extra weight pushes the search toward the goal. The book calls this a satisficing search: you accept a path that is good enough, not the shortest one. W = 10 is not a number from the book. I tried it because it found a path in a few seconds. Other weights were less stable in small tests.

Beam search with width k = 500 and k = 5000. At every depth I keep only the k boards with the smallest Manhattan value. This search is incomplete and not optimal, but it often finds a reasonably good solution and uses less memory than A*. The value of k is also not from the book. 500 is cheap. 5000 is wider, so it throws away fewer boards.

I did not use misplaced tiles as the search heuristic here. The book shows that Manhattan dominates it, so using both together is just Manhattan again.

Results

Strategy	Path	Generated	Expanded	Time
A* (g + h)	no (limit)	887,441	250,000	2.37 s
Weighted A* (W = 10)	1116	1,159,075	314,397	3.83 s
Beam k = 500	1078	1,996,339	535,239	4.45 s
Beam k = 5000	740	13,759,668	3,650,285	39.05 s

The shortest path I actually found is 740 moves, with beam k = 5000. It is not proven optimal. Manhattan says at least 306, so there is still a gap. I still keep this path because it is the best one among the solvers that finished. Weighted A* was faster but longer (1116). Beam k = 500 was in the middle (1078) and cheaper than k = 5000. So, if I only care about length, I would take k = 5000. If I also care about work, k = 500 is more balanced.

I want to be clear about A*. The heuristic is admissible, so it underestimates. That is exactly why A* is optimal, and also why it is slow. A more informed heuristic would still underestimate, but the gap to the true cost would be smaller, and A* would expand fewer nodes. Manhattan is good, but on a 9×9 board it is not strong enough for A* to finish on my laptop.

Portion of the explored tree

The full solution has hundreds of moves, so I cannot draw it. I generated the first two levels from the start with the real successor function (blank Up / Down / Left / Right). From the corner the blank can only go Up or Left. Each node in the figure is the 9×9 board. The yellow cell is the hole. Edge labels are the moves. I wrote this graph to a Graphviz .dot file and then rendered it.

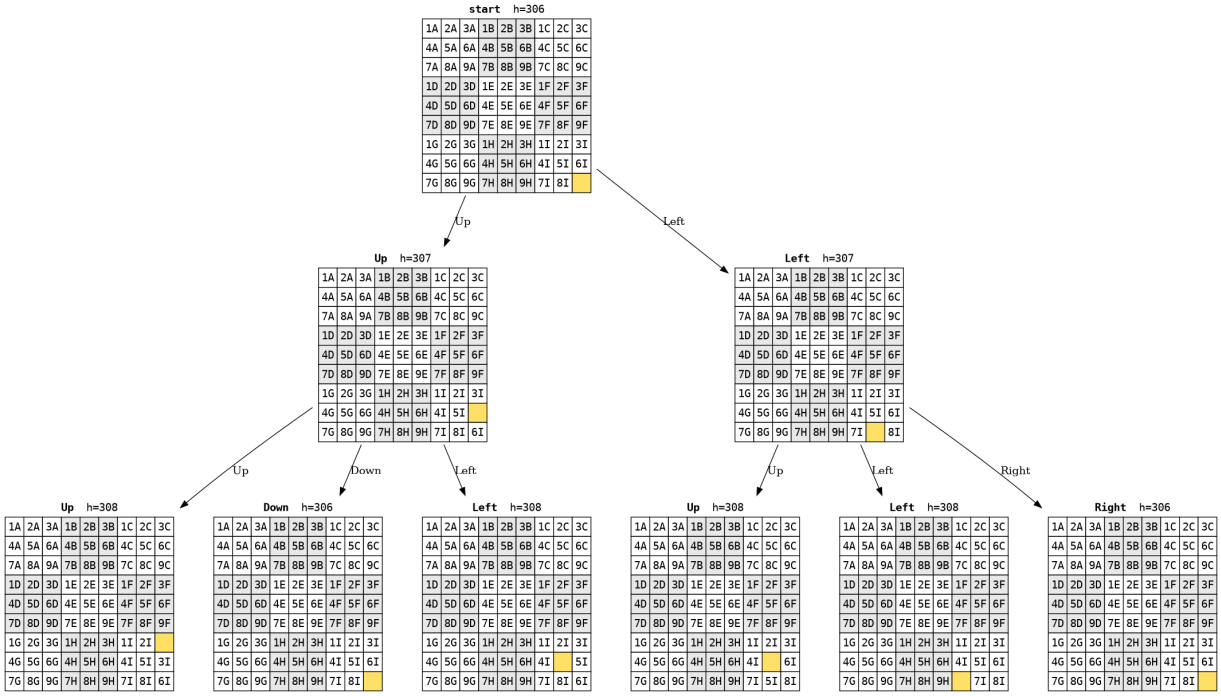


Figure 1. *Portion of the 80-tile search tree (depth 2) from the start board. h is Manhattan distance.*

One small detail that I found useful: after Up, h goes from 306 to 307, and after some children it goes back to 306. So the first move is not always “closer” to the goal. Greedy search can be fooled by this. A* would still keep both branches, because it also counts g.

3. N-Queens

This is not a shortest-path problem. I treated it as a constraint satisfaction problem, like Chapter 5. The variables are the columns. The domain of each column is the set of rows. The constraints are: no two queens in the same row, and no two queens on the same diagonal.

I used incremental backtracking. I place one queen per column, from left to right. Before I put a queen, I check the row and both diagonals on the left (the columns I already filled). If a row is not safe, I skip it. If I place a queen and later fail, I take it off and try the next row.

The program works for any N, not only 8. For N = 8 it finds this board:

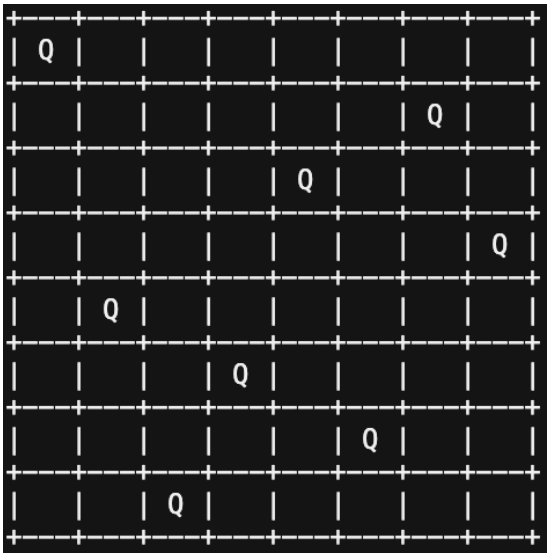


Figure 2. *Solution board for the N-Queens Problem with N = 8*

I do not claim this is the only solution, and I do not search for all of them. The idea was to solve the problem, not to count every arrangement. For N = 2 and N = 3 there is no solution, and the program reports that.

Is it optimal? The word “optimal” does not mean much here, because we do not have path cost. Any legal placement is a solution. Backtracking is complete: if a solution exists, this search will find one, because it tries every safe row in order. It is not the fastest possible N-queens solver. The book later talks about MRV and min-conflicts. I still kept plain backtracking because that is the algorithm the exercise asks for, and for N = 8 it is already instant.

4. Farmer, Wolf, Goat, and Cabbage

I modeled the state with four bits: farmer, wolf, goat, cabbage. True means “right bank”, False means “left bank”. At the start everybody is on the right. The goal is everybody on the left.

The actions are: the farmer goes alone, or takes the wolf, or the goat, or the cabbage. The farmer must travel with the passenger. A state is illegal if the wolf is alone with the goat, or the goat is alone with the cabbage.

The state space is tiny (16 possible tuples, and only some are legal). So I used BFS. I did not need a heuristic. Because every crossing has the same cost, BFS gives the shortest sequence (Russell & Norvig, 2021, Chapter 3). I got 7 steps:

1) Farmer takes goat
2) Farmer goes alone

3) Farmer takes wolf
4) Farmer takes goat (comes back)
5) Farmer takes cabbage
6) Farmer goes alone
7) Farmer takes goat

There is a twin solution that takes the cabbage in step 3 and the wolf later. It also has 7 steps. Both are optimal. I keep the one BFS found first. DFS could also solve this puzzle, but it might return a longer walk with extra back-and-forth, so BFS is the better choice here.

5. Missionaries and Cannibals

This puzzle is close to the farmer problem, but the state is different. I used: missionaries on the left, cannibals on the left, boat on the left. Start is (3, 3, True). Goal is (0, 0, False). The boat holds at most two people, so the loads are (1,0), (2,0), (0,1), (0,2) and (1,1).

A bank is illegal when there is at least one missionary there and the cannibals are more than the missionaries. I check both banks. This is the same safety idea as the wolf and the goat, only with numbers.

Again the graph is small, so I used BFS. Same reason as before: unit cost, so the first solution is the shortest. The program found 11 crossings:

1) 2 cannibals cross
2) 1 cannibal returns
3) 2 cannibals cross
4) 1 cannibal returns
5) 2 missionaries cross
6) 1 missionary and 1 cannibal return
7) 2 missionaries cross
8) 1 cannibal returns
9) 2 cannibals cross
10) 1 missionary returns
11) 1 missionary and 1 cannibal cross

11 is optimal for this classic instance. If I used DFS I would still reach the other side, but not always in 11 steps. I did not add a heuristic because uninformed BFS already finishes instantly.

6. Sudoku

I modeled Sudoku as a CSP, following Chapter 5. The variables are the empty cells. The domain of each variable is {1, …, 9}. The constraints are: all-different on every row, every column and every 3×3 box.

The basic solver is backtracking, the same idea as N-Queens. I pick an empty cell, try a legal digit, recurse, and undo if I fail. The program reads the board from a text file (0 means empty), so it can run on other puzzles too.

Besides naive backtracking (left to right, top to bottom, digits 1 to 9), I implemented three improvements from Section 5.3:

MRV (minimum remaining values): always fill the empty cell with the fewest legal digits. The book calls this fail-first. If some cell already has zero options, MRV picks it now and we backtrack at once.

LCV (least constraining value): try first the digit that removes the fewest options from the neighbors. This is about the order of values, not the order of cells.

Forward checking: after I write a digit, I look at the empty neighbors. If one of them has an empty domain, I backtrack immediately.

I report each improvement alone against naive backtracking, in order to compare the ideas. On these boards MRV already sees a dead cell very soon, so extra forward checking does not change the story much.

Board 1 (30 givens)

Version	Assignments	Backtracks	Max depth	Time
Naive backtracking	4,208	4,157	51	0.024 s
MRV	51	0	51	0.008 s
LCV	2,084	2,033	51	0.029 s
Forward checking	516	465	51	0.025 s

Board 2 (22 givens, harder)

Version	Assignments	Backtracks	Max depth	Time
Naive backtracking	335,637	335,578	59	2.09 s
MRV	4,036	3,977	59	1.01 s
LCV	341,523	341,464	59	5.07 s
Forward checking	153,330	153,271	59	7.73 s

MRV is the clear winner. On board 1 it solves the puzzle with 51 assignments and zero backtracks, which is basically “never guess wrong”. On board 2 it still needs some backtracks, but it is about 80 times fewer assignments than naive. This is the fail-first idea from the book: if you work on the tightest cell, you hit the conflict early and you do not build a big wrong tree.

Forward checking also helps naive search (516 vs 4,208 on board 1, and about half the assignments on board 2), because it notices empty domains sooner. It is slower in clock time on board 2, because each step does extra work on the neighbors. So fewer nodes does not always mean less time.

LCV alone did not help much. On the hard board it was even a bit worse than naive. LCV changes the order of digits, but if you still pick a bad cell first, you can still go down a huge branch. That is why the book usually pairs value ordering with a good variable ordering.

Max depth is 51 on board 1 and 59 on board 2. That is just the number of empty cells. Every version has to fill all of them, so the depth of a successful path is the same. The heuristics change the width of the tree, not this depth.

Brute force vs backtracking

If I ignore the constraints until the end, board 1 has 51 empty cells, so I could write 9^51 full grids in the worst case. Board 2 has 59 empty cells, so 9^59. These numbers are impossible in practice. Backtracking is better because it never writes a digit that already breaks a row, column or box, and it stops a branch as soon as a cell has no legal value. Most of that 9^51 tree is never built. MRV makes this pruning even earlier.

Finally, I found out that both boards have exactly one solution, which is what a well-posed Sudoku should have.

Short closing note

For small puzzles (farmer, missionaries, 8-tile) I trusted BFS when I needed the shortest path. For N-queens and Sudoku I used backtracking, because they are CSPs, not path costs. For the 80-tile, optimal A* did not finish, so I kept beam search with k = 5000 as my best found path (740 moves), and I still show A* as the algorithm that would be optimal if I had more time and a stronger heuristic.

References

Campbell, M., Hoane, A. J., Jr., & Hsu, F.-h. (2002). Deep Blue. Artificial Intelligence, 134(1–2), 57–83. https://doi.org/10.1016/S0004-3702(01)00129-1

Fraenkel, A. S., & Lichtenstein, D. (1981). Computing a perfect strategy for n × n chess requires time exponential in n. Journal of Combinatorial Theory, Series A, 31(2), 199–214. https://doi.org/10.1016/0097-3165(81)90016-9

Lichtenstein, D., & Sipser, M. (1980). GO is polynomial-space hard. Journal of the ACM, 27(2), 393–401. https://doi.org/10.1145/322186.322201

Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed., Global Edition). Pearson.

Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., & Hassabis, D. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. Science, 362(6419), 1140–1144. https://doi.org/10.1126/science.aar6404

van den Herik, H. J., Uiterwijk, J. W. H. M., & van Rijswijck, J. (2002). Games solved: Now and in the future. Artificial Intelligence, 134(1–2), 277–311. https://doi.org/10.1016/S0004-3702(01)00152-7